8 Embedded Applications

Embedded Linux Design and Programming

Raghav Vinjamuri

After the kernel has booted

- use the embedded Linux platform to create a working device for their applications.
- One approach is the using Python packaging and deployment options to assist with the iterative development of an application.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Chapter Objectives

In This Chapter

- Python and Docker – Basic Ideas
- Assignment
- Quiz

CHAPTER OVERVIEW

In This Chapter, We Will Discuss

1. Python and Docker – Basic Ideas
2. Assignment
3. Quiz

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Why Python?

Python is the most popular programming language for machine learning.

- Combine that with the proliferation of machine learning in our day-to-day lives and it is no surprise that the desire to run Python on edge devices is intensifying.
- Even in this era of transpilers and WebAssembly, packaging Python applications for deployment is an unsolved problem.
- what choices are out there for bundling Python modules together and when to use one method over another.
  - **https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf**

Python is the most popular programming language for machine learning.

- With proliferation of machine learning in day-to-day lives and it is no surprise that the desire to run Python on edge devices is intensifying.
- Even with transpilers and WebAssembly, packaging Python applications is an unsolved problem.
- what choices are there for bundling Python modules together and when to use one method over another.
- origins of today's Python packaging solutions, from the built-in standard distutils to its successor, setuptools.
- Next, is pip package manager, before reading about venv for Python virtual environments, followed by conda, the reigning general-purpose crossplatform solution.
- Lastly, the author will show how to use Docker to bundle Python applications along with their user space environment for rapid deployment to the cloud.
- Since Python is an interpreted language, a program cannot be compiled into a standalone executable like that with a language such as Go. This makes deploying Python applications complicated.
- Running a Python application requires installing a Python interpreter and several runtime dependencies.
- These requirements need to be code-compatible for the application to work.
- That requires the precise versioning of software components.
- Solving these deployment problems is what Python packaging is all about.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Using Ubuntu 20.04 or later

Using Ubuntu 20.04 or later

- The author recommends using Ubuntu 20.04 LTS or later for this chapter.
- Even though Ubuntu 20.04 LTS runs on the Raspberry Pi 4, the author still prefers to develop on an x86-64 desktop PC or laptop.
- Also the author chose Ubuntu for the development environment because the distribution maintainers keep Docker up to date.
- Ubuntu 20.04 LTS also comes with Python 3 and pip already installed since Python is used extensively throughout the system.
- Do not uninstall python3 or it will render Ubuntu unusable.

- https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Using Ubuntu 20.04 or later

- The author recommends using Ubuntu 20.04 LTS or later for this chapter.
- Even though Ubuntu 20.04 LTS runs on the Raspberry Pi 4, the author still prefers to develop on an x86-64 desktop PC or laptop.
- Also the author chose Ubuntu for the development environment because the distribution maintainers keep Docker up to date.
- Ubuntu 20.04 LTS also comes with Python 3 and pip already installed since Python is used extensively throughout the system.
- Do not uninstall python3 or it will render Ubuntu unusable.

IMPORTANT NOTE

- Do not install Miniconda until it gets to the section on conda because it interferes with the earlier pip exercises that rely on the system Python installation.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

venv

In Python

- The venv module provides support for creating lightweight "virtual environments" with their own site directories, optionally isolated from system site
- Each virtual environment has its own Python binary (which matches the version of the binary that was used to create this environment) and can have its own independent set of installed Python packages in its site directories.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

venv

- To install venv on Ubuntu, enter the following:
- $ sudo apt install python3-venv

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Docker

- Getting Docker

Getting to install Docker.

Getting Docker

- To install Docker on Ubuntu 20.04 LTS, do the following:
  1. Update the package repositories:
     $ sudo apt update
  2. Install Docker:
     $ sudo apt install docker.io
  3. Start the Docker daemon and enable it to start at boot time:
     $ sudo systemctl enable --now docker
  4. Add yourself to the docker group:
     $ sudo usermod -aG docker <username>
- Replace <username> in that last step with your username.
- The authors recommend creating a separate Ubuntu user account rather than using the default ubuntu user account, which is supposed to be reserved for administrative tasks.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging

- Python packaging landscape is a vast graveyard of failed attempts and abandoned tools.
- In researching this topic, the authors recommend looking at when the information was published and do not trust any advice that may be out of date.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

- Python packaging landscape is a vast graveyard of failed attempts and abandoned tools.
- In researching this topic, the authors recommend looking at when the information was published and do not trust any advice that may be out of date.
- Also, best practices around dependency management change often within the Python community and the recommended solution one year may be a broken nonstarter the next.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
distutils and setuptools

- Most Python libraries are distributed using distutils or setuptools, including all the packages found on the Python Package Index (PyPI).
- Both distribution methods rely on a setup.py project specification file that the package installer for Python (pip) uses to install a package.
- pip can also generate or freeze a precise list of dependencies after a project is installed.
- This optional requirements.txt file is used by pip in conjunction with setup.py to ensure that project installations are repeatable.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

- Most Python libraries are distributed using distutils or setuptools, including all the packages found on the Python Package Index (PyPI).
- Both distribution methods rely on a setup.py project specification file that the package installer for Python (pip) uses to install a package.
- pip can also generate or freeze a precise list of dependencies after a project is installed.
- This optional requirements.txt file is used by pip in conjunction with setup.py to ensure that project installations are repeatable.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
distutils

- distutils is the original packaging system for Python.
- setuptools has become its preferred replacement.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

distutils is the original packaging system for Python.
setuptools has become its preferred replacement.

distutils

- included in the Python standard library since Python 2.0.
- provides a Python package of the same name that can be imported by a setup.py script.
- Even though distutils still ships with Python, it lacks some essential features, so direct usage of distutils is now actively discouraged. setuptools has become its preferred replacement.
- While distutils may continue to work for simple projects, the community has moved on.
- Today, distutils survives mostly for legacy reasons.
- Many Python libraries were first published back when distutils was the only one in town.
- Porting them to setuptools now would take considerable effort and could break existing users.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
setuptools

- distutils is the original packaging system for Python.
- setuptools has become its preferred replacement.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

setuptools has become preferred replacement for distutils.

setuptools

- setuptools extends distutils adds support for complex constructs for easier distribution of larger applications.
- It has become the de facto packaging system within the Python community.
  Like distutils, setuptools offers a Python package that is imported into the setup.py script.
  Another effort called “distribute” was an ambitious fork of setuptools that eventually merged back into setuptools 0.7, cementing the status of setuptools as the definitive choice for Python packaging.
- setuptools introduced a CLI utility known as easy_install (now deprecated) and a Python package called pkg_resources for runtime package discovery and access to resource files.
- setuptools can also produce packages that act as plugins for other extensible packages (for example,
- frameworks and applications).
- do this by registering entry points in the setup.py script for the other overarching package to import.
- The term distribution means something different in the context of Python.
- A distribution is a versioned archive of packages, modules, and resource files used to distribute a release.
  A release is a versioned snapshot of a Python project taken at a given point in time.
  To make matters worse, the terms package and distribution are overloaded and often used interchangeably.
- For now, say a distribution is what is downloaded, and a package is the modules installed and imported.
- Cutting a release can result in multiple distributions, such as source distribution and one or more built distributions.
- There can be different built distributions for different platforms, such as one with a Windows installer.

setuptools has become preferred replacement for distutils.

setuptools

- The term “built distribution” means that no build step is required before installation.
- It does not necessarily mean precompiled.
- Some built distribution formats such as Wheel (.whl) exclude compiled Python files, for example.
- A built distribution containing compiled extensions is known as a binary distribution.
- An extension module is a Python module that is written in C or C++.
- Every extension module compiles down to a single dynamically loaded library, such as a shared object (.so) on Linux and a DLL (.pyd) on Windows.
- Contrast this with pure modules, which must be written entirely in Python.
- The Egg (.egg) built distribution format introduced by setuptools supports both pure and extension modules.
- a Python source code (.py) file compiles down to a bytecode (.pyc) file when the Python interpreter imports a module at runtime. It is how a built distribution format such as Wheel might exclude precompiled Python files.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
setup.py

- a setup.py script that setuptools can use to install the program.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

setup.py

- Say a small program is being developed in Python, maybe query a remote REST API and saves response data to a local SQL database. How to package program together with dependencies for deployment?
- Start by defining a setup.py script that setuptools can use to install the program.
- Deploying with setuptools is the first step toward more elaborate automated deployment schemes.
- Even if the program is small enough to fit comfortably inside a single module, chances are it won't stay that way for long. So say that the program consists of a single file named follower.py.

  $ tree follower/
  follower/
  follower.py
- Convert this module into a package by splitting follower.py up into three separate modules and placing them inside a nested directory also named follower:

  $ tree follower/
  follower/
  follower
  fetch.py
  __main__.py
  store.py

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

setup.py

- __main__.py module is where the program starts, so it contains mostly top-level, user-facing functionality.
- fetch.py module contains functions for sending HTTP requests to the remote REST API and the
- store.py module contains functions for saving response data to the local SQL database.
- To run this package as a script, pass the -m option to the Python interpreter:

  $ PYTHONPATH=follower python -m follower
- PYTHONPATH env variable points to the directory where a target project's package directories are located.
- The follower argument after the –m option tells Python to run the __main__.py module belonging to the follower package.
- Nesting package directories inside a project directory like this paves the way for a program to grow into a larger application made up ofmultiple packages each with its own namespace.
- With the pieces of the project all in their right place, create a minimal setup.py script that setuptools to package and deploy it:

```
from setuptools import setup
setup(
name='follower',
version='0.1',
packages=['follower'],
include_package_data=True,
install_requires=['requests', 'sqlalchemy']
)
```

- install_requires argument is a list of external dependencies installed automatically for a project to work at runtime.
- The versions of these dependencies needed or where to fetch them were not specified.
  The authors only asked for libraries that look and act like requests and sqlalchemy.
- Separating policy from implementation like this allows to easily swap out the official PyPI version of a dependency with something else in case it is needed to fix a bug or add a feature.
- Adding optional version specifiers to your dependency declarations is fine, but hardcoding distribution URLs within setup.py as dependency_links is wrong in principle.
- The packages argument tells setuptools what in-tree packages to distribute with a project release.
- Since every package is defined inside its own subdirectory of the parent project directory, the only package being shipped in this case is follower.
- Also, the authors are including data files along with their Python code in this distribution.
- To do that, set the include_package_data argument to True so that setuptools looks for a MANIFEST.in file and installs all the files listed there. So, here are the contents of the MANIFEST.in file:

```
include data/events.db
```

- If the data directory contained nested directories of data to be included, they could all be glob along with their contents using recursive include:

```
recursive-include data *
```

- Here is the final directory layout:

```
$ tree follower
follower
data
events.db
follower
fetch.py
__init__.py
store.py
MANIFEST.in
€ setup.py
```

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

setup.py

- setuptools excels at building and distributing Python packages that depend on other packages.
- It is able to do this thanks to features such as entry points and dependency declarations, which are simply unavailable in distutils.
- setuptools works well with pip and new releases of setuptools arrive on a regular basis.
- The Wheel build distribution format was created to replace the Egg format that setuptools originated.
- That effort has largely succeeded with the addition of a popular setuptools extension for building wheels and pip's great support for installing wheels.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
Installing Python packages with pip

- Python comes with a tool called pip to manage project dependencies.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Installing Python packages with pip

- Managing project dependencies is a tricky business.
- define project's dependencies in a setup.py script. But to install or upgrade or replace a dependency with a better one, or to decide when it is safe to delete a dependency no longer needed.
- Python comes with a tool called pip that can help, especially in the early stages of a project.
- The initial 1.0 release of pip was on April 4, 2011, around same time that Node.js and npm were started.
- Before it became pip, the tool was named pyinstall.
  pyinstall was created in 2008 as an alternative to easy_install, which came bundled with setuptools at the time. easy_install is now deprecated and setuptools recommends using pip instead.
- Since pip is included with the Python installer and you can have multiple versions of Python installed on your system (for example, 2.7 and 3.8), it helps to know which version of pip you are running:
  $ pip --version
- If no pip executable is found, that probably means it is on Ubuntu 20.04 LTS without Python 2.7 installed. That is fine - use pip3 for pip and python3 for python throughout the rest of this section:
  $ pip3 --version
- If there is python3 but no pip3 executable, then install it as shown on:
  $ sudo apt install python3-pip
- pip installs packages to a directory called site-packages.
- To find the location of your site-packages directory, run the following command:
  $ python3 -m site | grep ^USER_SITE

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
Installing Python packages with pip

- Python comes with a tool called pip to manage project dependencies.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Installing Python packages with pip

- Managing project dependencies is a tricky business.
- define project's dependencies in a setup.py script. But to install or upgrade or replace a dependency with a better one, or to decide when it is safe to delete a dependency no longer needed.
- Python comes with a tool called pip that can help, especially in the early stages of a project.
- The initial 1.0 release of pip was on April 4, 2011, around same time that Node.js and npm were started.
- Before it became pip, the tool was named pyinstall.
  pyinstall was created in 2008 as an alternative to easy_install, which came bundled with setuptools at the time. easy_install is now deprecated and setuptools recommends using pip instead.
- Since pip is included with the Python installer and you can have multiple versions of Python installed on your system (for example, 2.7 and 3.8), it helps to know which version of pip you are running:
  $ pip --version
- If no pip executable is found, that probably means it is on Ubuntu 20.04 LTS without Python 2.7 installed. That is fine - use pip3 for pip and python3 for python throughout the rest of this section:
  $ pip3 --version
- If there is python3 but no pip3 executable, then install it as shown on:
  $ sudo apt install python3-pip
- pip installs packages to a directory called site-packages.
- To find the location of your site-packages directory, run the following command:
  $ python3 -m site | grep ^USER_SITE

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Installing Python packages with pip

- IMPORTANT NOTE is that the pip3 and python3 commands are only required for Ubuntu 20.04 LTS or later, which no longer comes with Python 2.7 installed. However Most Linux distributions still come with pip and python executables, so use the pip and python commands, if needed.
- To get a list of packages already installed on your system, use this command:
  $ pip3 list
- pip is also, just another Python package, so you could use pip to upgrade itself, but don't do that, long term. consider virtual environments.
- To get a list of packages installed in the site-packages directory, use the following:
  $ pip3 list --user
- This list should be empty or much shorter than the list of system packages.
- In example project in last section. cd into parent follower directory where setup.py is located and run:
  $ pip3 install --ignore-installed --user .
- pip will use setup.py to fetch and install packages declared by install_requires to site-packages directory.
  –user option instructs pip to install packages to the site-packages directory rather than globally.
  --ignore-installed option forces pip to reinstall required packages already present on system to sitepackages
- Now list all the packages in the site-packages directory again:

```
$ pip3 list --user
Package Version
---------- ---------
certifi 2020.6.20
chardet 3.0.4
follower 0.1
idna 2.10
requests 2.24.0
SQLAlchemy 1.3.18
urllib3 1.25.10
```

- This time, it would show both requests and SQLAlchemy in the package list; and, to view details on the SQLAlchemy package just installed,
  $ pip3 show sqlalchemy
- The details shown contain the Requires and Required-by fields.
- Both are lists of related packages.
- Use the values in these fields and successive calls to pip show to trace the dependency tree of your project. But it's probably easier to pip install a command-line tool called **pipdeptree** and use that instead.
- When a Required-by field becomes empty, that is a good indicator that it is now safe to uninstall a package from a system. If no other packages depend on the packages in the deleted package's Requires field, then it's safe to uninstall those as well.
- To uninstall sqlalchemy using pip:
  $ pip3 uninstall sqlalchemy -y
- The trailing -y suppresses the confirmation prompt.
  To uninstall more than one package at a time, simply add more package names before the -y.
  The –user option is omitted here because pip is smart enough to uninstall from sitepackages first when a package is also installed globally.
- Sometimes when a package is needed, that serves some purpose or utilizes a particular technology, but you don't know the name of it.
- Either use pip to perform a keyword search against PyPI from the command line but that approach often yields too many results.
- It is much easier to search for packages on the PyPI website (https://pypi.org/search/), which allows filtering results by various classifiers.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
requirements.txt

- Requirements files allow you to specify exactly which packages and versions pip should install for your project.

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

requirements.txt

- "pip install" installs latest published version of a package, but often there is a need to install specific version of a package that works with the project's code. Eventually, project's dependencies will need upgrading.
- But, first how to use pip freeze to fix dependencies.
- Requirements files allow you to specify exactly which packages and versions pip should install for a project.
- By convention, project requirements files are always named **requirements.txt**.
- requirements file is a list of pip install arguments enumerating project's dependencies precisely versioned so that there are no surprises when the project is to be to rebuild and deployed. It is good practice to add a requirements.txt file to your project's repo in order to ensure reproducible builds.
- Returning to the follower project, now that we have installed all our dependencies and verified that the code works as expected, freeze the latest versions of the packages that pip installed for us.
- pip has a freeze command that outputs the installed packages along with their versions.
- redirect the output from this command to a requirements.txt file:
  $ pip3 freeze --user > requirements.txt
- with a requirements.txt file, people who clone the project can install all its dependencies using the -r option and the name of the requirements file:
  $ pip3 install --user -r requirements.txt

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

requirements.txt

- The autogenerated requirements file format defaults to exact version matching (==).
  For example, a line such as requests==2.22.0 tells pip that the version of requests to install must be exactly 2.22.0. There are other version specifiers to utilize in a requirements file, such as minimum version (>=), version exclusion (!=), and maximum version (<=).
  Version exclusion (!=) matches any version except the right-hand side.
  Minimum version (>=) matches any version greater than or equal to the right-hand side.
  Maximum version matches any version less than or equal to the right-hand side.
- combine multiple version specifiers in a single line using commas to separate them:
  requests >=2.22.0,<3.0
- default behavior when pip installs packages specified in a requirements file is to fetch them all from PyPI.
- override PyPI's URL (https://pypi.org/simple/) with that of an alternate Python package index by adding a line such as the following to the top of your requirements.txt file:
  --index-url http://pypi.mydomain.com/mirror
- The effort required to stand up and maintain own private PyPI mirror is not insubstantial. When all needed to do is fix a bug or add a feature to a project dependency, it makes more sense to override the package source instead of the entire package index.
- e.g.
  Version 4.3 of the Jetpack SDK for the NVIDIA Jetson Nano is based on Ubuntu's 18.04 LTS distribution. The Jetpack SDK adds extensive software support for the Nano's NVIDIA Maxwell 128 CUDA cores, such as GPU drivers and other runtime components. use pip to install a GPU-accelerated wheel for TensorFlow from NVIDIA's package index:
  $ pip install --user --extra-index-url
  https://developer.download.nvidia.com/compute/redist/jp/v43
  tensorflow-gpu==2.0.0+nv20.1
- The authors mentioned earlier how hardcoding distribution URLs inside setup.py is wrong.
  Use the -e argument form in a requirements file to override individual package sources:
  -e git+https://github.com/myteam/flask.git#egg=flask
- In this example, the author is instructing pip to fetch the flask package sources from the team's GitHub fork of pallets/flask.git. The -e argument form also takes a Git branch name, commit hash, or tag name:
  -e git+https://github.com/myteam/flask.git@master
  -e git+https://github.com/myteam/flask.git@51429
  30ef57e2f0ada00248bdaeb95406d18eb7c
  -e git+https://github.com/myteam/flask.git@v1.0
- Using pip to upgrade project dependencies to the latest versions published on PyPI is also straightforward:
  $ pip3 install --upgrade --user -r requirements.txt
- After verifying that installing with pip:requirements.txt using the latest versions of dependencies do not break the project, write them back out to the requirements file:
  $ pip3 freeze --user > requirements.txt
- Make sure that freezing did not overwrite any of the overrides or special version handling in your requirements file. Undo any mistakes and commit the updated requirements.txt file to version control.
- At some point, upgrading your project dependencies will result in code breaking.
  A new package release may introduce a regression or incompatibility with the project.
  The requirements file format provides syntax to deal with these situations.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

requirements.txt

- Let's say when using version 2.22.0 of requests in a project and version 3.0 is released.
- According to the practice of semantic versioning, incrementing the major version number indicates that version 3.0 of requests includes breaking changes to that library's API.
- So, then express thenew version requirements like this:
  requests ~= 2.22.0
- The compatible release specifier (~=) relies on semantic versioning.
  Compatible means greater than or equal to the right-hand side and less than the next version major number (for example, >= 1.1 and == 1.*).
- these same version requirements for requests are less ambiguously expressed as follows:
  requests >=2.22.0,<3.0
- These pip dependency management techniques work fine if a single Python project is developed at a time.
- But chances are the same machine is used to work on several Python projects at once, each potentially requiring a different version of the Python interpreter.
- The biggest problem with using only pip for multiple projects is that it installs all packages to the same user sitepackages directory for a particular version of Python.
- This makes it very hard to isolate dependencies from one project to the next.

- For this, pip combines well with Docker for deploying Python applications.
- Add pip to a Buildroot- or Yocto-based Linux image but that only enables quick onboard experimentation.
- A Python runtime package installer such as pip is ill-suited for Buildroot and Yocto environments where the entire contents of your embedded Linux image are defined at build time.
- pip works great inside containerized environments such as Docker where the line between build time and runtime is often blurry.
- the Python modules available in the meta-python layer and how to define a custom layer for an application.
- use the requirements.txt files generated by pip freeze to inform the selection of dependencies from meta-python for your own layer recipes.
- Buildroot and Yocto both install Python packages in a system-wide manner, so the virtual environment techniques discussed next do not apply to embedded Linux builds.

- They do, however, make it easier to generate accurate requirements.txt files.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging

Managing Python virtual environments with venv

A virtual environment is a self-contained directory tree containing
- a Python interpreter for a particular version of Python,
- a pip executable for managing project dependencies, and
- a local site-packages directory. .

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Managing Python virtual environments with venv

A virtual environment is a self-contained directory tree containing
- a Python interpreter for a particular version of Python,
- a pip executable for managing project dependencies, and
- a local site-packages directory.

- Switching between virtual environments tricks the shell into thinking that the only Python and pip executables available are the ones present in the active virtual environment. Best practice is to create a different virtual environment for each of your projects.
- This form of isolation solves the problem of two projects depending on different versions of same package.
- Virtual environments are not new to Python. The system-wide nature of Python installs mandates them.
- Besides enabling installing different versions of the same package, virtual environments also provide an easy way for you to run multiple versions of the Python interpreter.
- Several options exist for managing Python virtual environments.
  A tool that was immensely popular only 2 years ago (pipenv) has since languished.
  Meanwhile, a new contender has arisen (poetry) and Python 3's built-in support for virtual environments (venv) is starting to see more adoption.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Managing Python virtual environments with venv

- venv has been shipping with Python since version 3.3 (released in 2012).
- venv is incompatible with projects that require Python 2.7. Since support for Python 2.7 officially ended on January 1, 2020, Python 3 limitation is less of a concern.
- venv is based on the popular virtualenv tool, which is still maintained and available on PyPI.
- If there are projects that still require Python 2.7 for one reason or another, then use virtualenv instead of venv to work on those.
- By default, venv installs the most recent version of Python found on your system.
  With multiple versions of Python on a system, select a specific Python version by running python3 or whichever version needed when creating each virtual environment (The Python Tutorial, https://docs.python.org/3/tutorial/venv.html).
- Developing with the most recent version of Python is usually fine for greenfield projects but unacceptable for most legacy and enterprise software.
- Use the version of Python 3 that came with Ubuntu system to create and work with a virtual environment.
- To create a virtual environment, first decide where to put it, and then run venv module as a script with the target directory path:
  1. Ensure venv is installed on your Ubuntu system:
     $ sudo apt install python3-venv
  2. Create a new directory for your project:
     $ mkdir myproject
  3. Switch to that new directory:
     $ cd myproject
  4. Create the virtual environment inside a subdirectory named venv:
     $ python3 -m venv ./venv
- Now that a virtual environment is created, here is how to activate and verify it:
  1. Switch to your project directory if you haven't already:
     $ cd myproject
  2. Check where your system's pip3 executable is installed:
     $ which pip3
     /usr/bin/pip3
  3. Activate the project's virtual environment:
     $ source ./venv/bin/activate
  4. Check where your project's pip3 executable is installed:
     (venv) $ which pip3
     /home/frank/myproject/venv/bin/pip3
  5. List the packages that came installed with the virtual environment:
     (venv) $ pip3 list
     Package Version
     ---------- -------
     pip 20.0.2
     pkg-resources 0.0.0
     setuptools 44.0.0
- With the “which pip” command from within a virtual env, pip now points to the same executable as pip3.
- Prior to activating the virtual environment, pip probably did not point to anything because Ubuntu 20.04 LTS no longer comes with Python 2.7 installed.
- The same is true for python versus python3.
  Omit the 3 when running either pip or python from within a virtual environment.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Managing Python virtual environments with venv

- Next, let's install a property-based testing library named hypothesis into an existing virtual environment:
  1. Switch to your project directory if you haven't already:
     $ cd myproject
  2. Reactivate the project's virtual environment if it is not already active:
     $ source ./venv/bin/activate
  3. Install the hypothesis package:
     (venv) $ pip install hypothesis
  4. List the packages now installed inside the virtual environment:
     (venv) $ pip list
     Package Version
     ---------- -------
     attrs 19.3.0
     hypothesis 5.16.1
     pip 20.0.2
     pkg-resources 0.0.0
     setuptools 44.0.0
     sortedcontainers 2.2.2
- Notice that two new packages were added to the list besides hypothesis, attrs and sortedcontainers. hypothesis depends on these two packages.
- Say there was another Python project that depended on version 18.2.0 instead of version 19.3.0 of sortedcontainers.
- Those two versions would be incompatible and thus conflict with each other.
- Virtual environments allow installing both versions of the same package, a different version for each of the two projects.
- Also, switching out of a project directory does not deactivate its virtual environment.
- Deactivating a virtual environment is as easy as this:
  (venv) $ deactivate
  $
- This gets back in the global system environment where python3 and pip3 are needed again.
- This is everything needed to know to get started with Python virtual environments.

- Creating and switching between virtual environments is common practice now when developing in Python.
- Isolated environments make it easier to keep track of and manage your dependencies across multiple projects.

- Deploying Python virtual environments to embedded Linux devices for production makes less sense, but can still be done using a Debian packaging tool called dhvirtualenv (https://github.com/spotify/dh-virtualenv).

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging

Installing precompiled binaries with conda

conda is a package and virtual environment management system used by the Anaconda distribution of software for the PyData community.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Installing precompiled binaries with conda

conda is a package and virtual environment management system used by the Anaconda distribution of software for the PyData community.

- The Anaconda distribution includes Python as well as binaries for several hard-to-build open source projects such as PyTorch and TensorFlow.
- conda can be installed without the full Anaconda distribution, which is very large, or the minimal Miniconda distribution, which is still over 256 MB.
- Even though it was created for Python shortly after pip, conda has evolved into a general-purpose package manager like APT or Homebrew.
- Now, it can be used to package and distribute software for any language. Because conda downloads precompiled binaries, installing Python extension modules is a breeze.
- Another one of conda's big selling points is that it is cross-platform, with full support for Linux, macOS, and Windows.
- Besides package management, conda is also a full-blown virtual environment manager.
- Conda virtual environments have all the benefits we have come to expect from Python venv environments and more.
- Like venv, conda lets you use pip to install packages from PyPI into a project's local site-packages directory.
- If needed, use conda's own package management capabilities to install packages from different channels.
- Channels are package feeds provided by Anaconda and other software distributions.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
conda - Environment management

- Unlike venv, conda's virtual env. manager easily juggle multiple versions of Python, including Python 2.7.
- Need to have Miniconda installed on the Ubuntu system
- Use Miniconda instead of Anaconda for virtual environments because Anaconda environments come with lots of preinstalled packages, many of which are never needed.
- Miniconda environments are stripped down and allow easy install any of Anaconda's packages for.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Environment management

- Unlike venv, conda's virtual env. manager easily juggle multiple versions of Python, including Python 2.7.
- Need to have Miniconda installed on the Ubuntu system
- Use Miniconda instead of Anaconda for virtual environments because Anaconda environments come with lots of preinstalled packages, many of which are never needed.
- Miniconda environments are stripped down and allow easy install any of Anaconda's packages for.

  To install and update Miniconda on Ubuntu 20.04 LTS, do the following:

  1. Download Miniconda:
     $ wget
     https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
  2. Install Miniconda:
     $ bash Miniconda3-latest-Linux-x86_64.sh
  3. Update all the installed packages in the root environment:
     (base) $ conda update --all
- A fresh Miniconda installation comes with conda and a root environment containing a Python interpreter and some basic packages installed.
- By default, python and pip executables of conda's root environment are installed into the home directory.
- The conda root environment is known as base.
- View its location along with the locations of any other available conda environments by using command:
  (base) $ conda env list

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Environment management

- Verify this root environment before creating own conda environment:
    1. Open a new shell after installing Miniconda.
    2. Check where the root environment's python executable is installed:
       (base) $ which python
    3. Check the version of Python:
       (base) $ python --version
    4. Check where the root environment's pip executable is installed:
       (base) $ which pip
    5. Check the version of pip:
       (base) $ pip --version
    6. List the packages installed in the root environment:
       (base) $ conda list
- Next, create and work with own conda environment named py377:
    1. Create a new virtual environment named py377:
       (base) $ conda create --name py377 python=3.7.7
    2. Activate your new virtual environment:
       (base) $ source activate py377
    3. Check where your environment's python executable is installed:
       (py377) $ which python
    4. Check that the version of Python is 3.7.7:
       (py377) $ python --version
    5. List the packages installed in your environment:
       (py377) $ conda list
    6. Deactivate your environment:
       (py377) $ conda deactivate
- Using conda to create a virtual environment with Python 2.7 installed is as simple as the following:
  (base) $ conda create --name py27 python=2.7.17
- View conda environments again to see whether py377 and py27 now appear in the list:
  (base) $ conda env list
- Lastly, let's delete the py27 environment since it won't be used:
  (base) $ conda remove --name py27 â€“all

So now after knowing how to use conda to manage virtual environments, let's use it to manage packages within those environments.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python packaging
Package management

- as a general-purpose package manager, conda has its own facilities for managing dependencies:
- "conda list" lists all the packages that conda has installed in the active virtual environment.
- conda also uses of package feeds, which are called channels:

.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Package management

- Since conda supports virtual environments, use pip to manage Python dependencies from one project to another in an isolated manner just like with venv.
- As a general-purpose package manager, conda has its own facilities for managing dependencies:
  "conda list" lists all the packages that conda has installed in the active virtual environment.
  conda also uses of package feeds, which are called channels:
  1. get the list of channel URLs conda is configured to fetch from by entering this command:
     (base) $ conda info
  2. Before proceeding any further, let's reactivate the py377 virtual environment created earlier:
     (base) $ source activate py377
     (py377) $
  3. Most Python development happens inside a Jupyter notebook, so install those packages first:
     (py377) $ conda install jupyter notebook
  4. Enter y when prompted. It installs the jupyter and notebook packages along with all dependencies.
  conda list shows list of installed packages to be much longer than before. Now, install more packages :
     (py377) $ conda install opencv matplotlib
  5. Again, enter y when prompted. This time, the number of dependencies installed is smaller. Both opencv and matplotlib depend on numpy, so conda installs that package automatically without you having to specify it. If you want to specify an older version of opencv, you can install the desired version of the package this way:
     (py377) $ conda install opencv=3.4.1

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Package management

6. conda will then attempt to solve the active environment for this dependency. Since no other packages installed in this active virtual environment depend on opencv, the target version is easy to solve for. If they did, then it is a package conflict and the reinstallation would fail. After solving, conda prompts before downgrading opencv and its dependencies. Enter y to downgrade opencv to version 3.4.1.
7. Now if a newer version of opencv is released that addresses your previous concern. This is how to upgrade opencv to the latest version provided by the Anaconda distribution:

```
(py377) $ conda update opencv
```

8. This time, conda will prompt you to ask whether you want to update opencv and its dependencies for the latest version. This time, enter n to cancel the package update. Instead of updating packages individually, it's often easier to update all the packages installed in an active virtual environment at once:

```
(py377) $ conda update --all
```

9. Removing installed packages is also straightforward:

```
(py377) $ conda remove jupyter notebook
```

10. When conda removes jupyter and notebook, it removes all dangling dependencies as well. A dangling dependency is an installed package that no other installed packages depend on. Like most generalpurpose package managers, conda will not remove any dependencies that other installed packages still depend on.
11. Sometimes you may not know the exact name of a package to install. Amazon offers an AWS SDK for Python called Boto. Like many Python libraries, there is a version of Boto for Python 2 and a newer version (Boto3) for Python 3. To search Anaconda for packages with the word boto in their names, enter the following command:

```
(py377) $ conda search '*boto*'
```

12. boto3 and botocore are seen in the search results. At this time, the most recent version of boto3 available on Anaconda is 1.13.11. To view details on that specific version of boto3, enter:

```
(py377) $ conda info boto3=1.13.11
```

13. The package details reveal that boto3 version 1.13.11 depends on botocore (botocore >=1.16.11,<1.17.0), so installing boto3 gets you both.

- Now if all the packages needed to develop an OpenCV project inside a Jupyter notebook, are installed. how to share these project requirements with someone else so that they can recreate the work environment?
- The answer may surprise:

1. export active virtual environment to a YAML file:

```
(py377) $ conda env export > my-environment.yaml
```

2. Much like the list of requirements that pip freeze generates, the YAML that conda exports is a list of all the packages installed in a virtual environment together with their pinned versions. Creating a conda virtual environment from an environment file requires the -f option and the filename:

```
$ conda env create -f my-environment.yaml
```

3. The environment name is included in the exported YAML, so no --name option is necessary to create the environment. When creating a virtual environment from my-environment.yaml it will now show py377 in the list of environments with conda env list.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Package management

- conda is a very powerful tool in a developer's arsenal.
  By combining generalpurpose package installation with virtual environments, it offers a compelling deployment story.
- conda achieves many of the same goals Docker (up next) does, but without the use of containers.
- It has an edge over Docker with respect to Python due to its focus on the data science community.
- Because the leading machine learning frameworks (such as PyTorch and TensorFlow) are largely CUDA-based, finding GPU-accelerated binaries is often difficult. conda solves this problem by providing multiple precompiled binary versions of packages.
- Exporting conda virtual environments to YAML files for installation on other machines offers another deployment option. This solution is popular among the data science community, but it does not work in production for embedded Linux.

- conda is not one of the three package managers that Yocto supports.
  Even if conda was an option, the storage needed to accommodate Minconda on a Linux image is not a good fit for most embedded systems that are resource-constrained.
- If your dev board has an NVIDIA GPU such as the NVIDIA Jetson series, then use conda for onboard development.
- there is a conda installer named Miniforge (https://github.com/conda-forge/miniforge) that is known to work on 64-bit ARM machines like the Jetsons.

- With conda onboard, install jupyter, numpy, pandas, scikit-learn, and most of the other popular Python data science libraries out there.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

python

Deploying Python applications with Docker

- Docker offers another way to bundle Python code with software written in other languages:
- The idea behind Docker is that instead of packaging and installing application onto a preconfigured server environment, build and ship a container image with the application and all its runtime dependencies.
- A container image is more like a virtual environment than a virtual machine.
- A virtual machine is a complete system image including a kernel and an operating system.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Deploying Python applications with Docker

- Docker offers another way to bundle Python code with software written in other languages.
- The idea behind Docker is that instead of packaging and installing application onto a preconfigured server environment, build and ship a container image with the application and all its runtime dependencies.
- A container image is more like a virtual environment than a virtual machine.
- A virtual machine is a complete system image including a kernel and an operating system.
- A container image is a minimal user space environment that only comes with the binaries needed to run an application.
- Virtual machines run on top of a hypervisor that emulates hardware.
- Containers run directly on top of the host operating system.
- Unlike virtual machines, containers are able to share the same operating system and kernel without the use of hardware emulation.
- Instead, they rely on two special features of the Linux kernel for isolation: namespaces and cgroups.
- Docker did not invent container technology, but they were the first to build tooling that made them easy to use.
- The tired excuse of works on my machine no longer flies now that Docker makes it so simple to build and deploy container images.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Deploying Python applications with Docker

- Docker offers another way to bundle Python code with software written in other languages:
- The idea behind Docker is that instead of packaging and installing application onto a preconfigured server environment, build and ship a container image with the application and all its runtime dependencies.
- A container image is more like a virtual environment than a virtual machine.
- A virtual machine is a complete system image including a kernel and an operating system.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

The anatomy of a Dockerfile

- A Dockerfile describes the contents of a Docker image.
- Every Dockerfile contains a set of instructions specifying environment to use and which commands to run.
- Instead of writing a Dockerfile from scratch, use an existing Dockerfile for a project template.
- Dockerfile generates a Docker image for a simple Flask web application that can be extended to fit needs.
- Docker image is built on Alpine Linux, a slim Linux distribution commonly used for container deployments.
- Besides Flask, the Docker image also includes uWSGI and Nginx for better performance.
- Start at uwsgi-nginx-flask-docker project on GitHub (https://github.com/tiangolo/uwsgi-nginx-flask-docker).
- Then,click on the link to the python-3.8-alpine Dockerfile from the README.md file.
- Now look at the first line in that Dockerfile:
  FROM tiangolo/uwsgi-nginx:python3.8-alpine
- This FROM command tells Docker to pull an image named uwsgi-nginx from the tiangolo namespace with the python3.8-alpine tag from Docker Hub.
- Docker Hub is a public registry where people publish their Docker images for others to fetch and deploy.
- You can set up your own image registry using a service such as AWS ECR or Quay.
- Insert the name of your registry service in front of your namespace like this:
  FROM quay.io/my-org/my-app:my-tag
- Otherwise, Docker defaults to fetching images from Docker Hub.
- FROM is like an include statement in a Dockerfile. It inserts the contents of another Dockerfile into yours so that you have something to build on top of. The author likes to think of this approach as layering images.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

The anatomy of a Dockerfile

- Alpine is the base layer, followed by Python 3.8, then uWSGI plus Nginx, and finally your Flask application.
- learn more about how image layering works by digging into the python3.8- alpine Dockerfile at https://hub.docker.com/r/tiangolo/uwsgi-nginx.
- The next line of interest in the Dockerfile is the following:
  RUN pip install flask
- A RUN instruction runs a command. Docker executes the RUN instructions contained in the Dockerfile sequentially in order to build the resulting Docker image. This RUN instruction installs Flask into the system site-packages directory. pip is available because the Alpine base image also includes Python 3.8.
- Let's skip over Nginx's environment variables and go straight to copying:
  COPY ./app /app
- This particular Dockerfile is located inside a Git repo along with several other files and subdirectories.
- The COPY instruction copies a directory from the host Docker runtime environment (usually a Git clone of a repo) into the container being built.
- The python3.8-alpine.dockerfile file you are looking at resides in a docker-images subdirectory of the tiangolo/uwsgi-nginx-flaskdocker repo. Inside that docker-images directory is an app subdirectory containing a Hello World Flask web application. This COPY instruction copies the app directory from the example repo into the root directory of the Docker image:
  WORKDIR /app
- The WORKDIR instruction tells Docker which directory to work from inside the container. In this example, the /app directory that it just copied becomes the working directory. If the target working directory does not exist, then WORKDIR creates it. Any subsequent non-absolute paths that appear in this Dockerfile are hence relative to the /app directory.
- To set an environment variable inside the container:
  ENV PYTHONPATH=/app
- ENV tells Docker that what follows is an environment variable definition.
- PYTHONPATH is an environment variable that expands into a list of colon delimited paths where the Python interpreter looks for modules and packages.
- Next, let's jump a few lines down to the second RUN instruction:
  RUN chmod +x /entrypoint.sh
- The RUN instruction tells Docker to run a command from the shell. In this case, the command being run is chmod, which changes file permissions. Here it renders the /entrypoint.sh executable.
- The next line in this Dockerfile is optional:
  ENTRYPOINT ["/entrypoint.sh"]
- ENTRYPOINT is the most interesting instruction in this Dockerfile.
- It exposes an executable to the Docker host command line when starting the container and allows you to pass arguments from the command line down to the executable inside the container. append these arguments after docker run <image> on the command line.
- If there is more than one ENTRYPOINT instruction in a Dockerfile, then only the last one is executed.
- The last line in the Dockerfile is as follows:
  CMD ["/start.sh"]
- Like ENTRYPOINT instructions, CMD instructions execute at container start time rather than build time. When an ENTRYPOINT instruction is defined in a Dockerfile, a CMD instruction defines default arguments to be passed to that ENTRYPOINT.
- In this instance, the /start.sh path is the argument passed to /entrypoint.sh. The last line in /entrypoint.sh executes /start.sh:
  exec "$@"
- The /start.sh script comes from the uwsgi-nginx base image.
- /start.sh starts Nginx and uWSGI after /entrypoint.sh has configured the container runtime environment for them. When CMD is used in conjunction with ENTRYPOINT, the default arguments set by CMD can be overridden from the Docker host command line.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

The anatomy of a Dockerfile

- Most Dockerfiles do not have an ENTRYPOINT instruction, so the last line of a Dockerfile is usually a CMD instruction that runs in the foreground instead of default arguments.
- The author uses this Dockerfile trick to keep a general-purpose Docker container running for development:
  CMD tail -f /dev/null
- With the exception of ENTRYPOINT and CMD, all of the instructions in this example python-3.8-alpine Dockerfile only execute when the container is being built.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Building a Docker image

Building a Docker image
- Before building a Docker image, we need a Dockerfile.
- There may already be some Docker images on your system.
  To see a list of Docker images, use the following:
  $ docker images

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Building a Docker image
- Before building a Docker image, we need a Dockerfile.
- There may already be some Docker images on your system.
  To see a list of Docker images, use the following:
  $ docker images
- Now, let's fetch and build the Dockerfile just dissected:
  1. Clone the repo containing the Dockerfile:
     $ git clone https://github.com/tiangolo/uwsginginx-flask-docker.git
  2. Switch to the docker-images subdirectory inside the repo:
     $ cd uwsgi-nginx-flask-docker/docker-images
  3. Copy python3.8-alpine.dockerfile to a file named Dockerfile:
     $ cp python3.8-alpine.dockerfile Dockerfile
  4. Build an image from the Dockerfile:
     $ docker build -t my-image .
- Once the image is done building, it will appear in your list of local Docker images:
  $ docker images
- A uwsgi-nginx base image should also appear in the list along with the newly built my-image.
- Notice that the elapsed time since the uwsgi-nginx base image was created is much greater than the time since my-image was created.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Running a Docker image

Running a Docker image

- We now have a Docker image built that we can run as a container.
- To get a list of running containers on your system, use the following:

```
$ docker ps
```

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Running a Docker image

- We now have a Docker image built that we can run as a container.
  To get a list of running containers on your system, use the following:

```
$ docker ps
```

- To run a container based on my-image, issue the following docker run command:

```
$ docker run -d --name my-container -p 80:80 my-image
```

- Now observe the status of your running container:

```
$ docker ps
```

- It shows a container named my-container based on an image named my-image in the list.
  The -p option in the docker run command maps a container port to a host port.
  So, container port 80 maps to host port 80 in this example.
  This port mapping allows the Flask web server running inside the container to service HTTP requests.
- To stop my-container, run this command:

```
$ docker stop my-container
```

- Now check the status of your running container again:

```
$ docker ps
```

- my-container should no longer appear in the list of running containers. Is the container gone?
  No, it is only stopped. You can still see my-container and its status by adding the -a option to the docker ps command:

```
$ docker ps -a
```

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Fetching a Docker image

Fetching a Docker image

- It is much quicker to fetch the prebuilt image from Docker Hub than to build one yourself on your system.
- Docker images for the project can be found at https://hub.docker.com

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Fetching a Docker image

- Earlier in this section, the author touched on image registries such as Docker Hub, AWS ECR, and Quay.
- As it turns out, the Docker image that we built locally from a cloned GitHub repo is already published on Docker Hub.
- It is much quicker to fetch the prebuilt image from Docker Hub than to build it yourself on your system.
- The Docker images for the project can be found at https://hub.docker.com/r/tiangolo/uwsgi-nginx-flask.
- To pull the same Docker image that we built as my-image from Docker Hub, enter the following command:
  $ docker pull tiangolo/uwsgi-nginx-flask:python3.8-alpine
- Now look at your list of Docker images again:
  $ docker images
- You should see a new uwsgi-nginx-flask image in the list.
- To run this newly fetched image, issue the following docker run command:
  $ docker run -d --name flask-container -p 80:80 tiangolo/uwsgi-nginx-flask:python3.8-alpine
- substitute the full image name (repo:tag) in the preceding docker run command with the corresponding image ID (hash) from docker images if you prefer not to type out the full image name.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Publishing a Docker image

Publishing a Docker image

- publish a Docker image to Docker Hub.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Publishing a Docker image

- To publish a Docker image to Docker Hub, must first have an account and log in to it.
  Create an account on Docker Hub by going to the website, https://hub.docker.com and, signing up.
  Once you have an account, then you can push an existing image to your Docker Hub repository:
  1. Log in to the Docker Hub image registry from the command line:
     $ docker login
  2. Enter your Docker Hub username and password when prompted.
  3. Tag an existing image with a new name that starts with the name of your repository:
     $ docker tag my-image:latest <repository>/myimage: latest
     Replace <repository> with the name of your repository (the same as your username) on Docker Hub.
     Also substitute the name of another existing image you wish to push for myimage:latest.
  4. Push the image to the Docker Hub image registry:
     $ docker push <repository>/my-image:latest
     Again, make the same replacements as you did for Step 3.
- Images pushed to Docker Hub are publicly available by default. To visit web page for the newly published image, go to https://hub.docker.com/repository/docker/<repository>/my-image.
- Replace <repository> with the name of your repository (same as your username) on Docker Hub. You can also substitute the name of the actual image you pushed for my-image:latest if different.
- If you click on the Tags tab on that web page, you should see the docker pull command for fetching that image.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Cleaning up

Publishing a Docker image

- publish a Docker image to Docker Hub.

https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf

Cleaning up

- Using **docker images** lists images and **docker ps** lists containers.
- Before we can delete a Docker image, we must first delete any containers that reference it.
- To delete a Docker container, you first need to know the container's name or ID:
  1. Find the target Docker container's name:
     $ docker ps -a
  2. Stop the container if it is running:
     $ docker stop flask-container
  3. Delete the Docker container:
     $ docker rm flask-container
- Replace flask-container in the two preceding commands with the container name or ID from Step 1.
- Every container that appears under docker ps also has an image name or ID associated with it.
- Once all the containers that reference an image are deleted, you can then delete the image.
- Docker image names (repo:tag) can get long (for example, tiangolo/uwsgi-nginx-flask:python3.8-alpine).
- For that reason, the author finds it easier to just copy and paste an image's ID (hash) when deleting:
  1. Find the Docker image's ID:
     $ docker images
  2. Delete the Docker image:
     $ docker rmi <image-ID>
- Replace <image-ID> in the preceding command with the image ID from Step 1.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Cleaning up

- To simply blow away all the containers and images that you are no longer using on your system, then here is the command:

  $ docker system prune -a
- docker system prune deletes all stopped containers and dangling images.

- We've seen how pip can be used to install a Python application's dependencies.
- You simply add a RUN instruction that calls pip install to your Dockerfile.

- Because containers are sandboxed environments, they offer many of the same benefits that virtual environments do.

- But unlike conda and venv virtual environments, Buildroot and Yocto both have support for Docker containers.

- Buildroot has the docker-engine and docker-cli packages.
- Yocto has the meta-virtualization layer.

- If your device needs isolation because of Python package conflicts, then you can achieve that with Docker.

- The docker run command provides options for exposing operating system resources to containers.

- Specifying a bind mount allows a file or directory on the host machine to be mounted inside a container for reading and writing.

- By default, containers publish no ports to the outside world.
- When run from the my-container image, the -p option was used to publish port 80 from the container to port 80 on the host.
- The --device option adds a host device file under /dev to an unprivileged container.

- To grant access to all devices on the host, then use the --privileged option.
- What containers excel at is deployments.
- Being able to push a Docker image that can then be easily pulled and run on any of the major cloud platforms has revolutionized the DevOps movement.

- Docker is also making inroads in the embedded Linux space thanks to OTA update solutions such as balena.

- One of the downsides of Docker is the storage footprint and memory overhead of the runtime.
- The Go binaries are a bit bloated, but Docker runs on quad-core 64-bit ARM SoCs such as the Raspberry Pi 4 just fine.
- If your target device has enough power, then run Docker on it.
- Per the authors, the software development team will thank you.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Summary Notes

- **Understanding things is important .. to create a robust and maintainable embedded system**

1. So, what does any of this Python packaging stuff have to do with embedded Linux?
   The answer is not much, but bear in mind that the word programming also happens to be in the title of this book.
2. And this chapter has everything to do with modern-day programming.
3. To succeed as a developer in this day and age, you need to be able to deploy your code to production fast, frequently, and in a repeatable manner.
4. That means managing your dependencies carefully and automating as much of the process as possible.
5. You have now seen what tools are available for doing that with Python.
6. Understanding these things is important if you want to create a robust and maintainable embedded system.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Summary

Further reading

- The following resources have more information about the topics introduced in this chapter:
- Python Packaging User Guide, PyPA: https://packaging.python.org
- setup.py vs requirements.txt, by Donald Stufft: https://caremad.io/posts/2013/07/setup-vs-requirement
- pip User Guide, PyPA: https://pip.pypa.io/en/latest/user_guide/
- Poetry Documentation, Poetry: https://python-poetry.org/docs
- conda user guide, Continuum Analytics: https://docs.conda.io/projects/conda/en/latest/user-guide
- docker docs, Docker Inc.: https://docs.docker.com/engine/reference/commandline/docker

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Quiz

Canvas

- 10 Questions
- 45 Mins
- 6 Attempts
- Highest Retained

Quiz

Good Luck!

Assignment

1. using a docker container
2. Submit screenshot of install and run.

Assignment

1. using a docker container
2. Submit screenshot of install and run.

Summary

In This Chapter

- Python and Docker – Basic Ideas
- Assignment
- Quiz

CHAPTER OVERVIEW

In This Chapter, We Discussed

1. Python and Docker – Basic Ideas
2. Assignment
3. Quiz

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Review & Questions

- Review
- Q & A

Q&A